

Activity 1.

I want to build a scalable, data-driven Selenium automation framework.

Do not provide the implementation immediately. Instead, act as a Solution Architect and gather my requirements using the Socratic questioning technique.

Rules:

- * Ask exactly one question at a time.
- * Each question should provide multiple-choice options (A, B, C, D...) along with a brief explanation of each option.
- * Wait for my response before asking the next question.
- * Ask only the questions necessary to fully understand my requirements.
- * Do not make assumptions unless I explicitly ask for recommendations.

Once all the requirements are gathered:

1. Summarize my chosen requirements.
2. Explain the architecture and design decisions.
3. Provide the recommended project structure.
4. Implement the complete framework step by step with well-commented Java code.
5. Explain why each design choice was made and discuss possible alternatives.
6. Follow industry best practices for Selenium, TestNG, Maven, Page Object Model, and data-driven testing.

Scalable, Data-Driven Selenium Automation Framework

Solution Architecture, Design Decisions & Java Implementation

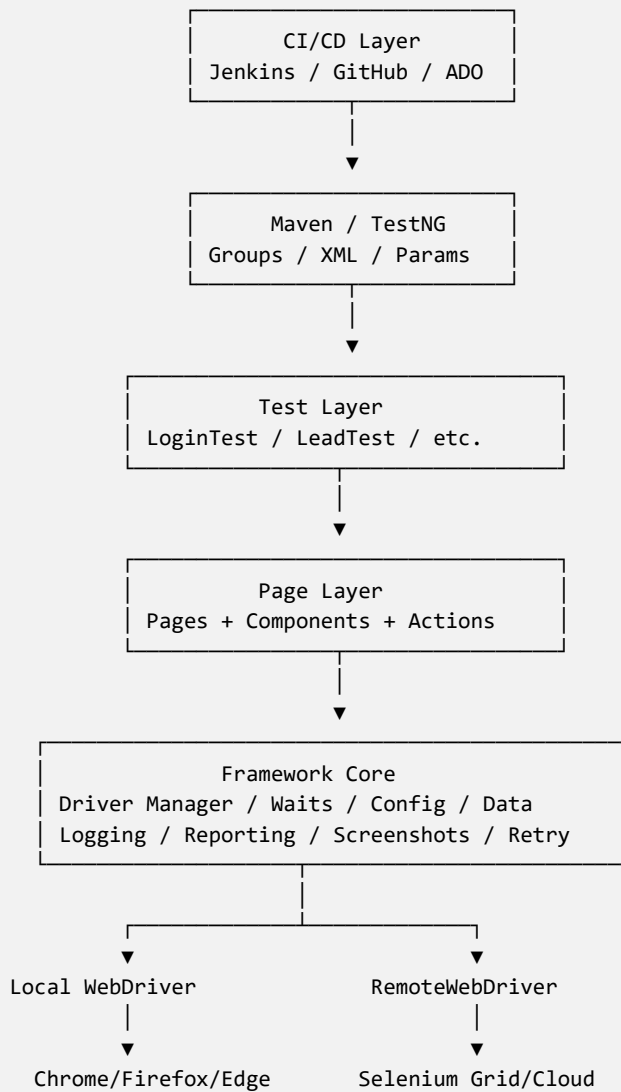
Prepared from the requirements gathered through a Socratic architecture discussion.

1. Finalized Requirements

Area	Selected Requirement
Application	Web application
Environments	DEV / QA / UAT
Browsers	Multiple browsers
Environment/browser selection	TestNG XML
Parallel execution	Browser + environment combination
WebDriver	Hybrid local + RemoteWebDriver
Driver provider	Fully configurable
Reporting	Enterprise-ready reporting layer
Failure evidence	Full artifact strategy
Test organization	TestNG groups
Retry	Retry + flaky-test tracking
CI/CD	Multiple platforms
Test isolation	Independent tests + controlled dependencies
Test data isolation	Enterprise hybrid
Java	8 / 11 / 17 / 21
POM	POM + reusable components + fluent actions
Locators	Dynamic locator strategy
Synchronization	Intelligent/dynamic synchronization
API	UI-first, API-ready
Secrets	Hybrid enterprise configuration
Data parameterization	One dataset per test
Browser capabilities	Fully dynamic capability provider

2. Recommended Architecture

The framework follows a layered architecture. Tests express business behavior, Page Objects/components encapsulate UI behavior, and framework services handle drivers, waits, configuration, data, reporting, logging, retries, and artifacts.



3. Recommended Project Structure

```
selenium-automation-framework/  
├── pom.xml  
├── testng/  
│   ├── smoke.xml  
│   ├── regression.xml  
│   ├── chrome-qa.xml  
│   ├── firefox-qa.xml  
│   └── cross-browser.xml  
├── src/  
│   ├── main/  
│   │   ├── java/com/kiran/automation/  
│   │   │   ├── config/  
│   │   │   │   ├── ConfigManager.java  
│   │   │   │   └── Environment.java  
│   │   │   └── driver/  
│   │   │       ├── DriverManager.java  
│   │   │       └── DriverFactory.java
```

```

├── Browser.java
├── DriverType.java
├── pages/
│   ├── BasePage.java
│   ├── LoginPage.java
│   └── DashboardPage.java
├── components/
│   ├── HeaderComponent.java
│   └── NavigationComponent.java
├── waits/WaitManager.java
├── data/
│   ├── TestData.java
│   ├── TestDataReader.java
│   └── TestDataProvider.java
├── listeners/
│   ├── TestListener.java
│   └── RetryAnalyzer.java
├── reporting/ReportManager.java
├── utils/
│   ├── ScreenshotUtils.java
│   ├── FileUtils.java
│   └── DateTimeUtils.java
├── constants/FrameworkConstants.java
├── resources/
│   ├── config/
│   │   ├── config.properties
│   │   ├── dev.properties
│   │   ├── qa.properties
│   │   └── uat.properties
│   ├── testdata/
│   │   ├── login.json
│   │   └── lead.json
│   └── logback.xml
├── test/java/com/kiran/tests/
│   ├── base/BaseTest.java
│   ├── smoke/LoginTest.java
│   └── regression/LeadTest.java
└── README.md

```

4. Maven Configuration

Maven should centrally manage Selenium, TestNG, logging, JSON parsing, and test execution dependencies. Dependency versions should be pinned in one place in the production project.

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.kiran</groupId>
  <artifactId>selenium-automation-framework</artifactId>
  <version>1.0.0</version>

```

```

<properties>
  <maven.compiler.source>8</maven.compiler.source>
  <maven.compiler.target>8</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<dependencies>

  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>${selenium.version}</version>
  </dependency>

  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>${testng.version}</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-api</artifactId>
    <version>${slf4j.version}</version>
  </dependency>

  <dependency>
    <groupId>ch.qos.logback</groupId>
    <artifactId>logback-classic</artifactId>
    <version>${logback.version}</version>
  </dependency>

  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>${jackson.version}</version>
  </dependency>

</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>${surefire.version}</version>
      <configuration>
        <suiteXmlFiles>
          <suiteXmlFile>
            testng/smoke.xml
          </suiteXmlFile>
        </suiteXmlFiles>
      </configuration>
    </plugin>
  </plugins>
</build>

</project>

```

5. TestNG Configuration

TestNG XML is the runtime entry point for environment and browser selection.

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">

<suite name="Chrome QA Suite">

    <parameter name="environment" value="qa"/>
    <parameter name="browser" value="chrome"/>
    <parameter name="executionMode" value="local"/>

    <test name="Smoke Tests">
        <groups>
            <run>
                <include name="smoke"/>
            </run>
        </groups>

        <packages>
            <package name="com.kiran.tests"/>
        </packages>
    </test>

</suite>
```

6. Configuration Manager

```
package com.kiran.automation.config;

import java.io.InputStream;
import java.util.Properties;

public class ConfigManager {

    private static final Properties properties =
        new Properties();

    static {
        try {
            InputStream inputStream =
                ConfigManager.class
                    .getClassLoader()
                    .getResourceAsStream(
                        "config/config.properties");

            if (inputStream == null) {
                throw new RuntimeException(
                    "config.properties not found");
            }

            properties.load(inputStream);

        } catch (Exception e) {
            throw new RuntimeException(
                "Unable to load configuration", e);
        }
    }
}
```

```

    }

    private ConfigManager() {
    }

    public static String get(String key) {

        String value = System.getProperty(key);

        if (value != null) {
            return value;
        }

        return properties.getProperty(key);
    }
}

```

7. Browser and WebDriver Management

```

package com.kiran.automation.driver;

public enum Browser {

    CHROME,
    FIREFOX,
    EDGE;

    public static Browser fromString(String browser) {
        return Browser.valueOf(
            browser.trim().toUpperCase());
    }
}

package com.kiran.automation.driver;

import org.openqa.selenium.WebDriver;

public final class DriverManager {

    private static final ThreadLocal<WebDriver>
        DRIVER = new ThreadLocal<>();

    private DriverManager() {
    }

    public static void setDriver(WebDriver driver) {
        DRIVER.set(driver);
    }

    public static WebDriver getDriver() {

        WebDriver driver = DRIVER.get();

        if (driver == null) {
            throw new IllegalStateException(
                "WebDriver has not been initialized");
        }
    }
}

```

```

        return driver;
    }

    public static void quitDriver() {

        WebDriver driver = DRIVER.get();

        if (driver != null) {
            driver.quit();
            DRIVER.remove();
        }
    }
}

package com.kiran.automation.driver;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.edge.EdgeDriver;

public class DriverFactory {

    public static WebDriver createLocalDriver(
        Browser browser) {

        switch (browser) {

            case CHROME:
                return new ChromeDriver();

            case FIREFOX:
                return new FirefoxDriver();

            case EDGE:
                return new EdgeDriver();

            default:
                throw new IllegalArgumentException(
                    "Unsupported browser: " + browser);
        }
    }
}

```

ThreadLocal is essential for your parallel execution requirement. Each TestNG execution thread gets an isolated WebDriver.

8. RemoteWebDriver Support

```

URL gridUrl = new URL(
    "http://localhost:4444/wd/hub");

WebDriver driver =
    new RemoteWebDriver(
        gridUrl,

```

```
capabilities);
```

The final `DriverFactory` should route to `LocalDriverProvider`, `RemoteDriverProvider`, or a future `CloudDriverProvider` according to configuration.

9. Base Page and Wait Manager

```
package com.kiran.automation.waits;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public class WaitManager {

    private final WebDriverWait wait;

    public WaitManager(WebDriver driver) {

        this.wait =
            new WebDriverWait(
                driver,
                Duration.ofSeconds(20));
    }

    public WebElement waitForVisible(By locator) {

        return wait.until(
            ExpectedConditions.visibilityOfElementLocated(
                locator));
    }

    public WebElement waitForClickable(By locator) {

        return wait.until(
            ExpectedConditions.elementToBeClickable(
                locator));
    }

    public boolean waitForUrl(String url) {

        return wait.until(
            ExpectedConditions.urlToBe(url));
    }
}

package com.kiran.automation.pages;

import com.kiran.automation.driver.DriverManager;
import com.kiran.automation.waits.WaitManager;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
```

```

import org.openqa.selenium.WebElement;

public abstract class BasePage {

    protected WebDriver driver;
    protected WaitManager wait;

    protected BasePage() {

        this.driver = DriverManager.getDriver();
        this.wait = new WaitManager(driver);
    }

    protected void click(By locator) {

        wait.waitForClickable(locator).click();
    }

    protected void type(By locator, String text) {

        WebElement element =
            wait.waitForVisible(locator);

        element.clear();
        element.sendKeys(text);
    }

    protected String getText(By locator) {

        return wait.waitForVisible(locator)
            .getText();
    }
}

```

10. Page Object Model + Fluent Actions

```

package com.kiran.automation.pages;

import org.openqa.selenium.By;

public class LoginPage extends BasePage {

    private final By username =
        By.id("username");

    private final By password =
        By.id("password");

    private final By loginButton =
        By.id("login");

    public LoginPage enterUsername(
        String usernameValue) {

        type(username, usernameValue);
        return this;
    }
}

```

```

    public LoginPage enterPassword(
        String passwordValue) {

        type(password, passwordValue);
        return this;
    }

    public DashboardPage clickLogin() {

        click(loginButton);
        return new DashboardPage();
    }
}

package com.kiran.automation.pages;

import org.openqa.selenium.By;

public class DashboardPage extends BasePage {

    private final By dashboardTitle =
        By.cssSelector("h1.dashboard-title");

    public boolean isDisplayed() {

        return wait
            .waitForVisible(dashboardTitle)
            .isDisplayed();
    }
}

```

11. Reusable Page Components

```

package com.kiran.automation.components;

import org.openqa.selenium.By;

public class HeaderComponent {

    private final By profile =
        By.id("profile");

    private final By logout =
        By.id("logout");

    public void logout() {
        // Reusable logout implementation.
    }
}

```

Use components for repeated UI structures such as headers, navigation menus, tables, dialogs, filters, and reusable widgets. This prevents giant Page Object classes.

12. Dynamic Locator Strategy

```
public By customer(String customerName) {  
  
    return By.xpath(  
        "//td[normalize-space()='"  
        + customerName  
        + "']");  
}
```

Preferred locator priority:

- data-testid / dedicated automation ID
- Stable unique ID
- Stable name or accessible attribute
- CSS selector
- XPath only where necessary

Avoid brittle absolute XPath such as /html/body/div[2]/div[3]/div[1]. Dynamic should mean parameterized and stable, not fragile.

13. Base Test

```
package com.kiran.tests.base;  
  
import com.kiran.automation.driver.Browser;  
import com.kiran.automation.driver.DriverFactory;  
import com.kiran.automation.driver.DriverManager;  
  
import org.openqa.selenium.WebDriver;  
import org.testng.annotations.*;  
  
public abstract class BaseTest {  
  
    @Parameters({"environment", "browser"})  
    @BeforeMethod(alwaysRun = true)  
    public void setUp(  
        String environment,  
        String browser) {  
  
        Browser selectedBrowser =  
            Browser.fromString(browser);  
  
        WebDriver driver =  
            DriverFactory  
                .createLocalDriver(  
                    selectedBrowser);  
  
        DriverManager.setDriver(driver);  
  
        driver.manage()  
            .window()  
            .maximize();  
  
        // Resolve the environment URL
```

```

        // using ConfigManager.
    }

    @AfterMethod(alwaysRun = true)
    public void tearDown() {
        DriverManager.quitDriver();
    }
}

```

14. Example Test

```

package com.kiran.tests.smoke;

import com.kiran.automation.pages.DashboardPage;
import com.kiran.automation.pages.LoginPage;
import com.kiran.tests.base.BaseTest;

import org.testng.Assert;
import org.testng.annotations.Test;

public class LoginTest extends BaseTest {

    @Test(groups = {"smoke"})
    public void validLoginTest() {

        LoginPage loginPage =
            new LoginPage();

        DashboardPage dashboardPage =
            loginPage
                .enterUsername("admin")
                .enterPassword("password")
                .clickLogin();

        Assert.assertTrue(
            dashboardPage.isDisplayed(),
            "Dashboard should be displayed");
    }
}

```

15. TestNG Groups

```

@Test(groups = {"smoke"})
public void validLogin() {
}

@Test(groups = {"regression"})
public void invalidPassword() {
}

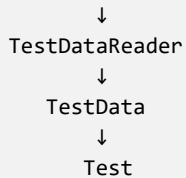
@Test(groups = {"smoke", "regression"})
public void anotherLoginScenario() {
}

```

16. Data-Driven Testing and Data Sources

Your selected approach is one dataset per test, while the framework remains capable of reading multiple external sources. The recommended abstraction is:

Excel / JSON / CSV / Database



```
public class TestData {  
  
    private String username;  
    private String password;  
  
    public String getUsername() {  
        return username;  
    }  
  
    public String getPassword() {  
        return password;  
    }  
  
    public void setUsername(String username) {  
        this.username = username;  
    }  
  
    public void setPassword(String password) {  
        this.password = password;  
    }  
}
```

17. Failure Evidence and Reporting

On failure, capture:

- Screenshot
- Page source
- Current URL
- Browser and environment
- Timestamp and test name
- Browser console logs where available
- Relevant test-data/context metadata
- Attachments to the reporting layer

```
package com.kiran.automation.utils;  
  
import com.kiran.automation.driver.DriverManager;  
  
import org.openqa.selenium.OutputType;  
import org.openqa.selenium.TakesScreenshot;
```

```

import java.io.File;
import java.nio.file.Files;
import java.nio.file.StandardCopyOption;

public class ScreenshotUtils {

    public static String capture(String testName) {

        try {

            File source =
                ((TakesScreenshot)
                 DriverManager.getDriver())
                 .getScreenshotAs(
                     OutputType.FILE);

            File destination =
                new File(
                    "target/screenshots/"
                    + testName
                    + ".png");

            destination.getParentFile().mkdirs();

            Files.copy(
                source.toPath(),
                destination.toPath(),
                StandardCopyOption.REPLACE_EXISTING);

            return destination.getAbsolutePath();

        } catch (Exception e) {
            return null;
        }
    }
}

```

18. TestNG Listener

```

public class TestListener
    implements ITestListener {

    @Override
    public void onTestFailure(
        ITestResult result) {

        String testName =
            result.getName();

        ScreenshotUtils.capture(testName);

        // Capture page source
        // Capture URL
        // Capture browser information
        // Attach artifacts to reporting layer
    }
}

```

```
}
```

19. Retry and Flaky-Test Tracking

```
public class RetryAnalyzer
    implements IRetryAnalyzer {

    private int retryCount = 0;
    private static final int MAX_RETRY = 1;

    @Override
    public boolean retry(
        ITestResult result) {

        if (retryCount < MAX_RETRY) {
            retryCount++;
            return true;
        }

        return false;
    }
}
```

A production implementation should distinguish transient infrastructure failures from functional assertion failures. Repeated FAIL → PASS patterns should be reported as potentially flaky rather than silently converted into clean passes.

20. Parallel Execution

```
<suite name="Cross Browser"
    parallel="tests"
    thread-count="3">

    <test name="QA Chrome">
        <parameter name="environment" value="qa"/>
        <parameter name="browser" value="chrome"/>
        <classes>
            <class name="com.kiran.tests.smoke.LoginTest"/>
        </classes>
    </test>

    <test name="QA Firefox">
        <parameter name="environment" value="qa"/>
        <parameter name="browser" value="firefox"/>
        <classes>
            <class name="com.kiran.tests.smoke.LoginTest"/>
        </classes>
    </test>

    <test name="UAT Chrome">
        <parameter name="environment" value="uat"/>
        <parameter name="browser" value="chrome"/>
        <classes>
            <class name="com.kiran.tests.smoke.LoginTest"/>
        </classes>
    </test>
</suite>
```

```
</test>

</suite>
```

The desired model is Thread 1 → QA Chrome, Thread 2 → QA Firefox, Thread 3 → UAT Chrome, with an isolated ThreadLocal driver for each execution.

21. CI/CD Strategy

Keep the Java framework CI/CD-tool-agnostic. Jenkins, GitHub Actions, and Azure DevOps should invoke Maven and publish standard reports/artifacts.

```
mvn clean test

mvn clean test -DsuiteXmlFile=testng/regression.xml
```

Typical CI artifacts:

- Test reports
- Screenshots
- Logs
- Page source
- Console/browser logs
- Test result XML

22. Logging

```
private static final Logger logger =
    LoggerFactory.getLogger(LoginPage.class);

logger.info("Entering username");
logger.debug("Username locator: {}", username);
logger.error("Login failed", exception);
```

Use SLF4J as the logging API and Logback as an implementation. Avoid `System.out.println` for framework-level diagnostics.

23. Secrets and Configuration

Never commit passwords, tokens, API keys, or other secrets to source control. Use configuration files for non-sensitive defaults and environment variables/CI secret stores for sensitive values.

```
String password =
    System.getenv("TEST_PASSWORD");
```

The configuration abstraction can later support enterprise secret stores such as Azure Key Vault, AWS Secrets Manager, or HashiCorp Vault.

24. API-Ready Architecture

```
automation-framework/  
├── ui/  
│   ├── pages/  
│   ├── components/  
│   └── driver/  
├── api/  
│   ├── clients/  
│   ├── requests/  
│   └── responses/  
├── data/  
├── config/  
├── reporting/  
└── utils/
```

Keep API testing separate from Page Objects. Later, API setup can create data quickly and UI tests can validate the user-facing result.

25. Test Independence and Data Isolation

Tests should establish their own prerequisites wherever practical. Use controlled TestNG dependencies only for genuine business dependencies.

```
public class DataGenerator {  
  
    public static String uniqueEmail() {  
  
        return "test_"  
            + System.currentTimeMillis()  
            + "@example.com";  
    }  
}
```

Each test should clean up only the records it created. Avoid shared mutable test data that can collide during parallel execution.

26. Architecture Decision Summary

Decision	Reason
ThreadLocal	Required for isolated WebDriver instances during parallel execution.
Driver Factory	Supports local, remote, and future cloud execution without changing tests.
Page Components	Prevents duplication for shared UI elements and keeps large applications maintainable.
TestNG Groups	Provides flexible smoke/regression/sanity classification as requested.
TestNG XML	Provides runtime browser/environment selection as requested.
Centralized Configuration	Prevents environment-specific logic from spreading across test classes.

Explicit/Intelligent Waits	Avoids brittle fixed sleeps and improves synchronization.
Failure Artifacts	Provides the evidence needed to debug CI and parallel failures.
Retry + Flaky Tracking	Retries transient failures without hiding recurring instability.
Reporting Abstraction	Allows the reporting implementation to evolve without changing test code.
API-ready separation	Allows future API setup/validation without restructuring the UI framework.

27. Recommended Adjustments

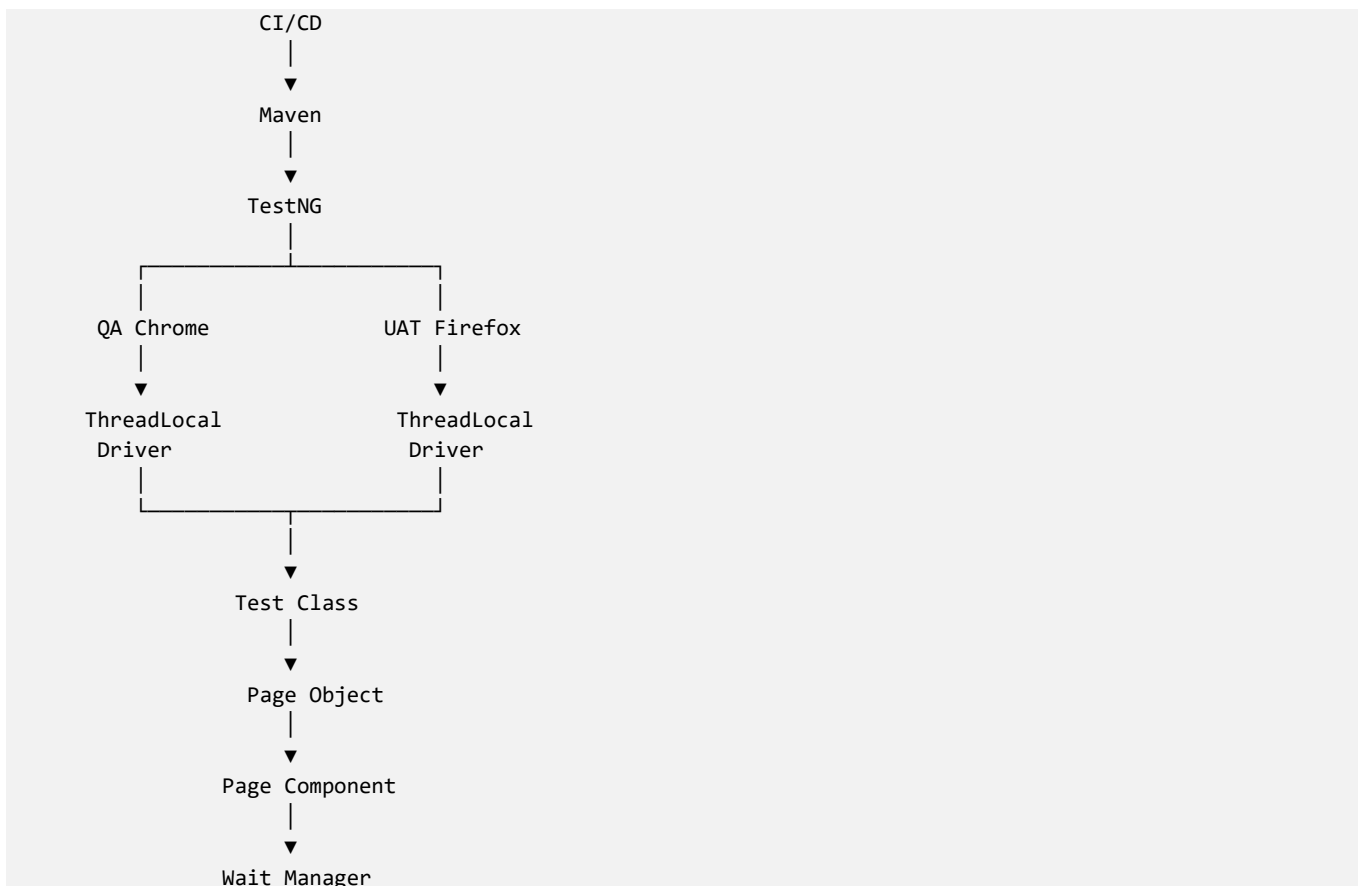
Java baseline

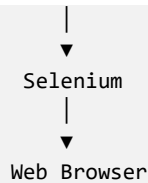
Although Java 8/11/17/21 compatibility was selected, a modern enterprise project should preferably establish Java 17 as the production baseline and optionally validate Java 21. Supporting Java 8 can constrain current Selenium and dependency choices.

Dynamic locators

Do not dynamically generate every locator. Prefer stable automation attributes, then parameterized locators, and use dynamic XPath only where necessary.

28. Complete Execution Flow





29. Failure Flow



30. Implementation Roadmap

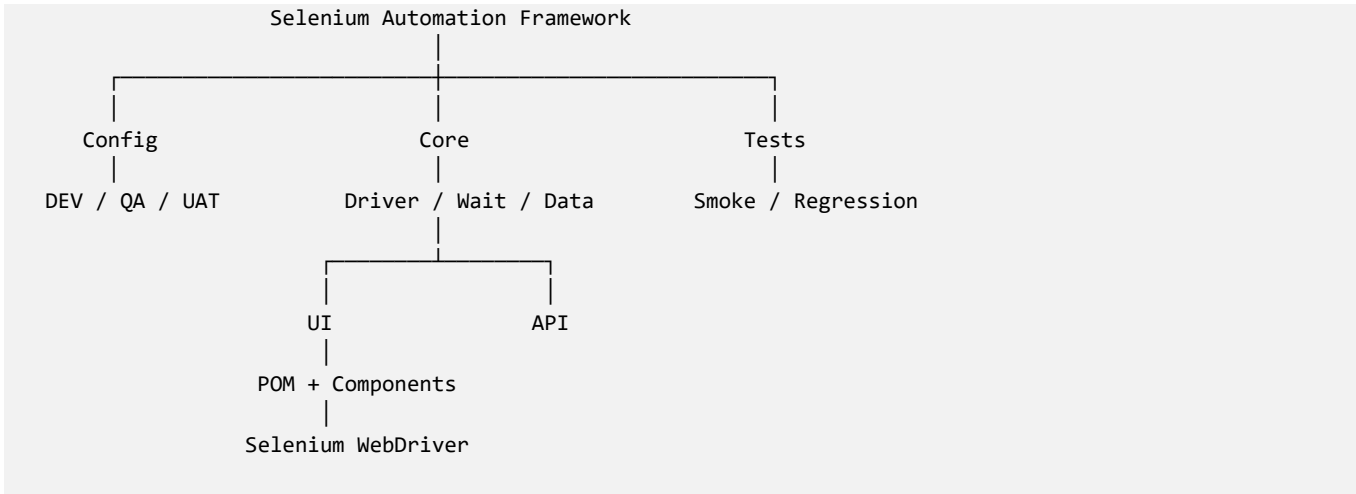
1. Maven + project foundation
2. Configuration management
3. Browser/driver abstraction
4. ThreadLocal + parallel execution
5. BasePage + intelligent waits
6. POM + reusable components
7. TestNG groups + suite XML
8. Test-data abstraction
9. Listeners + screenshots + failure artifacts
10. Retry + flaky-test tracking
11. Reporting abstraction
12. CI/CD + Selenium Grid + final enterprise hardening

31. Key Industry Best Practices

- Keep tests focused on business behavior and assertions.
- Keep Selenium implementation inside Page Objects/components/framework services.
- Never share a WebDriver across parallel threads.
- Avoid Thread.sleep(); use explicit, fluent, and application-aware synchronization.
- Prefer stable test IDs and accessible attributes for locators.
- Keep tests independent wherever practical.
- Generate unique runtime test data when parallel tests could collide.
- Do not store secrets in source control.
- Use listeners for cross-cutting concerns such as screenshots, reporting, and failure evidence.
- Keep CI/CD integration outside the core Java framework.

- Publish reports and failure artifacts from every CI run.
- Track flaky behavior instead of masking it with unlimited retries.
- Keep API automation as a separate layer even when it shares configuration and data services.

32. Final Architecture at a Glance



This document represents the architecture and starter implementation derived from the requirements selected during the Socratic design session. The code snippets are intentionally structured as a foundation; production implementation should pin verified dependency versions, add comprehensive error handling, complete remote capability creation, implement the selected reporting backend, and add CI/CD pipeline definitions.

Activity 2

Customer tries to transfer ₹50,000.

Transaction failed.

Error shown: "Transaction could not be completed."

Expected discoveries:

1. Daily limit exceeded
2. Insufficient balance
3. Bank server unavailable
4. OTP expired
5. Network timeout
6. Beneficiary inactive

- 7. Fraud rule triggered
- 8. Core banking unavailable
- 9. UPI switch unavailable

Transaction Failure Investigation

Chain of Thought Assignment – Structured Investigation

1. Scenario

Customer attempts to transfer ₹50,000. The transaction fails and the application displays the message:
"Transaction could not be completed."

2. Investigation Approach

The error message is generic, so the investigation should proceed through the transaction flow and verify each possible failure point.

3. Possible Root Causes

#	Possible Root Cause	What to Verify
1	Daily limit exceeded	Check the customer's daily transfer limit and the amount already transferred today.
2	Insufficient balance	Verify that the available account balance is at least ₹50,000 plus any applicable charges.
3	Bank server unavailable	Check bank/payment service availability, service health, and relevant server logs.
4	OTP expired	Verify whether the OTP was entered within its validity period.
5	Network timeout	Check request/response timeout between the customer application and backend services.
6	Beneficiary inactive	Verify beneficiary status, activation status, and transaction eligibility.
7	Fraud rule triggered	Check whether the

		transaction was blocked by fraud or risk-detection rules.
8	Core banking unavailable	Verify connectivity and availability between the transaction system and the core banking system.
9	UPI switch unavailable	If the transaction uses UPI, verify UPI switch/service availability and related errors.

4. Transaction Investigation Flow

Customer → Mobile/Web App → Authentication/OTP → Beneficiary Validation → Transaction/Limit Validation → Fraud/Risk Check → Payment/UPI Switch → Core Banking → Bank

At each stage, verify whether the request was successfully processed, rejected, timed out, or unavailable.

5. Expected Conclusion

The message "Transaction could not be completed" is a generic error and does not identify the actual root cause. Therefore, the QA or production support team should trace the transaction using the transaction ID/reference number, application logs, API responses, and backend/service logs.

The investigation should determine exactly which condition caused the transaction to fail rather than assuming the first possible cause.

6. Key QA Point

Do not assume the first possible cause is the actual cause. Each dependency and validation point should be independently verified using the appropriate logs, API responses, service health information, and transaction details.